

Informe de Auditoría Integral — ObSave v1.0.35

Ad Astra Forge · Plugin ObSave
Fecha: 4 de septiembre de 2026
Alcance: Persistencia OAuth, motor de sync, temporizador auto-sync, rendimiento
Nota: Análisis sin release asociado.

Resumen ejecutivo

Área	Veredicto	Severidad principal
Persistencia OAuth	El refresh_token sí se persiste y hay refresh silencioso; los síntomas de "re-auth" suelen deberse a errores no manejados (401, invalid_grant, carrera pendingConfig)	Media-Alta
Motor de sync	Un solo método canónico (sync()), pero con 4 wrappers y bloqueo no atómico	Media
Temporizador / desconexión	Timer bien gestionado; desalineación UI ↔ motor explica sync en background con carpeta "no lista"; badges no pasan a rojo al desconectar	Media
Rendimiento	Lectura + hash de todas las notas en cada ciclo aunque no haya cambios; inventario Drive completo cada 15 s por defecto	Alta

1. Auditoría de persistencia y re-autenticación (OAuth Session Lifecycle)

1.1 ¿Por qué parece perder el token o pide conectar de nuevo?

No hay lógica que borre tokens por tiempo o por desconexión de red. La sesión solo se elimina con Desconectar explícito (`disconnectProvider()` → `providerConfig.gdrive = null`).

Los síntomas de "sesión perdida" suelen provenir de:

Causa	Mecanismo	Efecto en UI
Refresh fallido (invalid_grant, red caída)	refreshAccessToken() lanza error; el timer solo hace console.warn	UI sigue "Conectado" (mira refreshToken en disco); sync falla sin guía de re-auth
Access token expirado + sin red	ensureValidAccessToken() no puede renovar	Error en sync; tokens intactos en data.json
Carrera pendingConfig	Tras refresh, GoogleDriveLazyProvider puede hacer connect(stalePendingConfig) antes de que termine saveSettings()	Sobrescritura de tokens recién renovados en memoria → 401 transitorios
Sin retry en 401	Solo refresh proactivo por expiresAt; no hay "401 → refresh → reintentar request"	Fallo aunque el token sea renovable

OAuth inicial sin refresh_token	Google a veces no devuelve refresh token sin prompt=consent + revocar acceso previo	Error explícito al conectar, no al usar
--	---	---

1.2 ¿Se almacena correctamente el refresh_token ?

Sí. Persistencia en Obsidian `data.json` vía `loadData / saveData` en `main.ts`.

Claves relevantes en `providerConfig.gdrive` :

- `refreshToken` — credencial persistente (define "conectado")
- `accessToken` , `expiresAt` — token de corta duración
- `folderId` , `folderPath` , `folderName` , `folderMode` , `folderSelected` — carpeta destino (**independiente** del OAuth)

"Conectado" = presencia de `refreshToken` (`isProviderConfigured` / `hasProviderCredentials`).

Tras OAuth PKCE, se escribe en dos rutas (redundante pero funcional):

1. `GoogleDriveProvider.persistConfig()` → listener en `main.ts` → `saveSettings()`
2. `onAuthSuccess` en `ObSaveSettingTab` → `saveSettings()` de nuevo

1.3 ¿Existe renovación silenciosa (refreshAccessToken)?

Sí, **completa y sin navegador** en `GoogleDriveProvider.ts` :

- POST a `oauth2.googleapis.com/token` con `grant_type=refresh_token`
- Margen proactivo: **5 minutos** antes de expiración (`TOKEN_REFRESH_MARGIN_MS`)
- Timer en background: `scheduleBackgroundRefresh()` con `setTimeout`
- El navegador solo interviene en el flujo PKCE inicial (`authenticateWithPkce`)

1.4 ¿Se borran folderId o sesión sin [Desconectar]?

No, salvo desconexión explícita. Operaciones que **no** borran sesión:

- Error de sync / refresh fallido
- Cambio de carpeta (picker)
- Sync exitoso (refuerza `folderId` y `folderSelected: true`)
- Cierre/reapertura de Obsidian

Desconectar sí limpia todo en `disconnectProvider()` :

- `stopAutoSync()`
- `providerConfig[active] = null`
- `activeProvider = null`
- `syncedLedger = {}`
- `autoSyncEnabled = false`

1.5 Bug estructural: carrera pendingConfig vs refresh

Cada operación lazy re-aplica `pendingConfig` **potencialmente obsoleto** en `GoogleDriveLazyProvider.ts` :

```
async sync(): Promise<SyncResult> {
  const delegate = await this.ensureLoaded();
  if (this.pendingConfig) {
    await delegate.connect(this.pendingConfig);
  }
}
```

```
}  
...  
}
```

Tras un refresh silencioso, `persistConfig` actualiza `settings` en disco, pero `pendingConfig` solo se sincroniza cuando termina `saveSettings()` → `applyPendingGoogleDriveConfig()`. Si hay sync/API entre medias, se puede **sobrescribir** tokens recién renovados.

Propuestas (persistencia)

1. **Sincronizar `pendingConfig` inmediatamente** en el listener de `onConfigChanged`, no solo al final de `saveSettings()`.
2. **Retry 401**: wrapper HTTP que haga `refreshAccessToken()` + reintento una vez.
3. **Detectar `invalid_grant`**: limpiar flag de sesión inválida y mostrar CTA "Reconectar" sin borrar `folderId` (modo "refresh session").
4. **Unificar escritura OAuth** en un solo path post-PKCE para evitar sobrescrituras de campos de carpeta.
5. **GitHub**: PAT en texto plano en `data.json`; sin refresh OAuth (comportamiento esperado, documentar riesgo local).

2. Unificación del motor de sincronización (Single Entry Point)

2.1 Mapa de entry points actuales

Origen	Ruta	Trigger final
Ribbon	<code>syncEngine.executeSync()</code>	"manual"
Dashboard	<code>plugin.runSync() → triggerSync(true)</code>	"manual"
Timer	<code>setInterval(() => sync("automatic"))</code>	"automatic"
Comandos palette	No implementados	—

Conclusión: no hay dos motores distintos; hay **un motor** (`sync()`) con **wrappers redundantes** y semántica duplicada en plugin vs engine.

2.2 Bloqueo actual y race conditions

En `SyncEngine.sync()`:

```
if (this.status === "syncing") {  
  return;  
}
```

- Flag implícito: `this.status === "syncing"` (no hay `isSyncing` explícito ni mutex)
- **TOCTOU**: entre el check y `setStatus("syncing")` hay ~50 líneas síncronas; dos llamadas simultáneas (ribbon + timer a 15 s) **pueden** pasar el guard
- Solapamiento **manual durante auto**: manual retorna sin hacer nada, pero el ribbon ya mostró "ObSave: Iniciando sincronización..." → UX engañosa
- Solapamiento **auto durante manual**: auto descartado silenciosamente (correcto para background, sin métrica)

2.3 Doble fuente de verdad de estado

- **Runtime:** `SyncEngine.status` (privado)
- **Persistido:** `settings.syncStatus` (actualizado vía eventos en `main.ts`)

Si Obsidian cierra mid-sync, `settings.syncStatus` puede quedar "syncing" mientras el engine arranca en "idle" → ribbon desincronizado hasta el primer evento.

Propuesta: `executeUnifiedSync(trigger: 'manual' | 'auto')`

```
interface SyncRunResult {
  ran: boolean;
  skippedReason?: "already-syncing" | "not-configured" | "gdrive-no-folder";
}

private syncInFlight: Promise<void> | null = null;

async executeUnifiedSync(trigger: SyncTrigger): Promise<SyncRunResult> {
  if (this.syncInFlight) {
    if (trigger === "manual") {
      this.emit({ type: "sync-skipped", reason: "already-syncing", trigger });
    }
    return { ran: false, skippedReason: "already-syncing" };
  }
  this.syncInFlight = this.doSync(trigger).finally(() => {
    this.syncInFlight = null;
  });
  await this.syncInFlight;
  return { ran: true };
}
```

Refactor recomendado:

Eliminar / deprecar	Reemplazar
<code>SyncEngine.executeSync()</code>	<code>executeUnifiedSync("manual")</code>
<code>ObSavePlugin.triggerSync()</code> (sin callers)	Eliminar
<code>ObSavePlugin.runSync()</code>	Delegar a <code>executeUnifiedSync("manual")</code>
Timer	<code>void this.executeUnifiedSync("automatic")</code>
<code>executeSyncGoogleDrive</code> (privado)	Renombrar → <code>runGoogleDriveBidirectionalSync</code>

Extras:

- Notice del ribbon **condicionado** a `result.ran === true`
- Al `onload`, resetear `settings.syncStatus` si era "syncing"
- Cola opcional: un auto-sync diferido al terminar manual (evitar perder ticks en intervalos cortos)

3. Ciclo de vida del temporizador y desconexión manual

3.1 Gestión del timer — correcta en lo esencial

En `SyncEngine.ts` :

- `startAutoSync()` : siempre llama `stopAutoSync()` primero; crea `setInterval` solo si `canAutoSync()`
- `stopAutoSync()` : `clearInterval` + `autoSyncIntervalId = null`
- `restartAutoSync()` : stop + start

Invocaciones:

Evento	Acción
onload	<code>startAutoSync()</code>
onunload	<code>stopAutoSync()</code>
<code>saveSettings()</code>	<code>restartAutoSync()</code>
<code>disconnectProvider()</code>	<code>stopAutoSync()</code> primero

No hay fugas obvias de `setInterval` si se pasa por estos paths.

3.2 ¿Por qué sync en background con carpeta "no seleccionada" en UI?

Desalineación de criterios:

Capa	Condición "carpeta lista"
UI (<code>folderReady</code>)	<code>folderSelected === true && !!folderId</code>
Motor (<code>canAutoSync</code>)	Solo <code>folderSelected === true</code> (GDrive)

Escenario concreto (modo "Crear carpeta nueva"):

Tras conectar en `ObSaveSettingTab` : `folderSelected = true` , pero `folderId` vacío hasta la primera sync.

- UI: `folderReady = false` → mensaje "Se creará la carpeta... en la primera sincronización"
- Motor: `canAutoSync() = true` → **timer activo**
- Si `autoSyncEnabled = true` , el intervalo (default **15 s**) dispara sync y crea la carpeta vía `getOrCreateTargetFolder()`

Esto explica la inconsistencia reportada: no es un timer zombie; es **auto-sync intencional** según el motor, mientras la UI muestra carpeta pendiente.

En modo "carpeta existente" sin seleccionar: `folderSelected = false` → UI bloquea toggle y `canAutoSync() = false` → timer no arranca.

3.3 ¿Qué hace [Desconectar] hoy?

Requisito solicitado	Estado actual
<code>stopAutoSync() + clearInterval</code>	✓

Borrado tokens + folderSelected = false	✅ vía providerConfig.gdrive = null (objeto entero eliminado)
Badges → todos rojos	❌ No implementado

En `FileStatusDecorator.getMarkdownFileStatuses()` : con `activeProvider = null` devuelve mapa vacío → `applyDecorations` **elimina todos los dots**; no los pinta de rojo.

3.4 Bugs adicionales en desconexión

Carrera sync-en-vuelo vs disconnect:

- `stopAutoSync()` no cancela sync activa
- Una sync que termine **después** de disconnect puede emitir `sync-complete` y restaurar `lastSyncAt` / reescribir `syncedLedger`

Sync automática silenciosa sin carpeta:

Si `folderSelected !== true`, sync automática retorna silenciosamente (sin evento). Si el timer siguiera activo con settings corruptos, seguiría disparando ticks inútiles.

Propuestas (timer / desconexión)

1. **Alinear criterios:** `canAutoSync()` debe usar la misma regla que `folderReady`, o documentar explícitamente que modo "new" permite auto-sync pre- `folderId`.
2. **Estado disconnected en decorador:** cuando `!isProviderConfigured()`, marcar **todas** las `.md` como "new" (rojo) o nuevo estado "disconnected".
3. **Token de cancelación en SyncEngine:** `disconnectProvider()` incrementa `syncGeneration`; sync en curso ignora resultados si la generación cambió.
4. **Auto-stop defensivo:** si `sync("automatic")` retorna por falta de carpeta/proveedor, llamar `stopAutoSync()` y logear.
5. **Reset SyncEngine.status** en disconnect.

4. Auditoría de rendimiento y consumo de recursos

4.1 Comparación de notas — trabajo innecesario

Problema crítico en `syncBothPresent` : lectura y hash **antes** del early exit:

```
const localContent = await this.app.vault.read(localFile);
const localHash = hashContent(localContent);
// ...
if (localMatchesLedger && remoteMatchesLedger) {
  return { action: "none", ... };
}
```

Para *N* notas estables en cada ciclo (cada **15 s** por defecto): **N lecturas de disco + N hashes**, aunque no haya cambios.

Contraste: el decorador **sí** usa atajo por `mtime` antes de leer.

Inventario remoto completo cada sync:

- `listAllMarkdownFiles` → walk recursivo: **~2 requests HTTP por carpeta**

- Sin Changes API ni caché incremental
- Con 100 carpetas y auto-sync 15 s → ~800 requests/min solo de inventario

Doble lectura en push: `syncBothPresent` lee el archivo; `pushLocalFile` vuelve a leer.

Metadatos Drive incompletos: `listFiles` no pide `size` → imposible filtro barato `mtime+size` antes de leer.

4.2 Hash — aclaración importante

No es SHA-256. Es **djb2** (32 bits, hex) en `contentHash.ts` :

- O(n) siempre; barato vs SHA-256, pero se invoca en demasiados archivos
- Colisiones posibles en bóvedas muy grandes (32 bits)

4.3 syncedLedger — memoria y crecimiento

- 1 entrada por nota sincronizada; serializado entero en `data.json` en cada `saveSettings()` / `sync-complete`
- Estimación: ~80–150 bytes JSON/nota → ~1–1.5 MB para 10 000 notas
- **Entradas huérfanas:** paths en ledger que ya no existen local ni remoto **no se podan**
- Falta `size` y `remoteModifiedTimeMs` para atajos de comparación

4.4 Refresco DOM — frecuencia excesiva

Disparador	Debounce
<code>layout-change</code>	400 ms (<code>requestRefresh</code>)
<code>vault.create</code> / <code>vault.modify</code>	Inmediato (cada keystroke en <code>.md</code>)
<code>sync-complete</code>	Immediate + <code>notifyVisualRefresh()</code> → otro ciclo vía <code>layout-change</code>

Cada refresco: `clearDecorations()` + `querySelectorAll` en todo el file-explorer + recálculo de **todas** las notas markdown.

Doble refresco post-sync: `refreshDecoratorsImmediate()` + `workspace.trigger("layout-change")` .

Propuestas de optimización (priorizadas)

P0 — Mayor impacto inmediato

1. Reordenar `syncBothPresent` : comparar `mtime` + `size` (+ `remoteMtime` + `driveFileId`) antes de `vault.read` + hash.
2. Extender `SyncLedgerEntry` : añadir `size` y `remoteMtime` .
3. Pedir `size` en Drive API: `fields=files(id,name,modifiedTime,mimeType,size)` .
4. Evitar doble lectura en push: pasar `content` ya leído a `pushLocalFile` .

P1 — Escalabilidad

5. Changes API de Drive (`changes.list` + `startPageToken`) en lugar de walk completo.
6. Poda del ledger al final de cada sync (eliminar paths huérfanos).
7. Persistir ledger solo si cambió (no en cada sync "sin cambios").
8. Eliminar `notifyVisualRefresh()` → `layout-change` si ya hay `refreshDecoratorsImmediate()` .
9. Debounce en `vault.modify` (400–800 ms) para edición activa.

P2 — Refinamiento

- 10. **Decorador incremental:** solo actualizar paths que cambiaron vs caché anterior.
- 11. **Intervalo default:** subir de 15 s a ≥5 min para bóvedas grandes, o backoff adaptativo si `noChanges` .
- 12. **Resolver ID post-upload** desde body de respuesta, no `listFiles(parent)` .

Matriz de coste estimado (sync sin cambios, N notas, F carpetas)

Operación	Actual	Con optimización mtime/size
API Drive (inventario)	~2F requests/ciclo	~2F / ~1-few con Changes API
<code>vault.read</code>	N (todas las notas en ambos lados)	0 si metadatos coinciden
<code>hashContent</code>	N	0 en caso estable
Persistencia <code>data.json</code>	Cada <code>sync-complete</code>	Solo si ledger cambió
Refresco DOM	<code>layout-change</code> + <code>post-sync</code>	Debounced + incremental

Plan de refactorización estructural recomendado

Fase A — Correcciones de consistencia (bajo riesgo)

- Fix carrera `pendingConfig` `post-refresh`
- Alinear `canAutoSync` con `folderReady` (o documentar modo `"new"`)
- Badges rojos / estado `disconnected` al desconectar
- Token de cancelación en `disconnect`
- Reset `syncStatus` al cargar plugin

Fase B — Motor unificado (medio riesgo)

- `executeUnifiedSync(trigger)` con mutex `syncInFlight`
- Eliminar wrappers muertos (`triggerSync` , duplicación `ribbon/dashboard`)
- Evento `sync-skipped` para UX coherente
- Retry 401 en capa HTTP Drive

Fase C — Rendimiento (mayor impacto, más pruebas)

- Atajo `mtime/size` en `sync` y decorador
- Ledger extendido + poda
- Changes API Drive (`sync incremental`)
- Reducir refrescos DOM

Respuestas directas a las inconsistencias detectadas

#	Pregunta	Respuesta
1	¿Por qué pierde token / pide re-auth?	No se borra automáticamente. Síntomas por <code>refresh</code> fallido, 401 sin <code>retry</code> , carrera <code>pendingConfig</code> , o <code>invalid_grant</code> sin flujo de reconexión guiado

1b	¿Refresh token persistido + refresh silencioso?	Sí , en data.json y vía refreshAccessToken() + timer 5 min antes de expirar
1c	¿folderId/sesión se borran sin Desconectar?	No , salvo desconexión explícita
2	¿Flujos duplicados manual vs auto?	Un motor (sync()), múltiples wrappers ; bloqueo no atómico
2b	Propuesta unificada	executeUnifiedSync(trigger) + syncInFlight mutex
3	¿Sync con carpeta "no seleccionada"?	Desalineación UI (folderId requerido) vs motor (folderSelected basta) en modo carpeta nueva
3b	¿Desconectar cumple checklist?	Timer  , tokens  ; badges rojos  (desaparecen); sync en vuelo no cancelada 
4	¿Llamadas/lecturas innecesarias?	Sí : inventario Drive completo + read/hash de todas las notas cada ciclo
4b	¿Ledger y DOM?	Ledger crece sin poda; DOM se refresca en cada layout-change y keystroke

Archivos clave auditados

Archivo	Rol
src/engine/SyncEngine.ts	Motor de sync, timer, ledger
src/providers/GoogleDriveProvider.ts	OAuth, refresh, API Drive
src/providers/GoogleDriveLazyProvider.ts	Carga diferida, carrera pendingConfig
src/main.ts	Plugin, ribbon, disconnect, eventos
src/ui/ObSaveSettingTab.ts	Dashboard, OAuth UI, auto-sync toggle
src/ui/FileStatusDecorator.ts	Badges en explorador
src/utils/contentHash.ts	Hash djb2
src/settings.ts	Esquema settings y ledger

Documento generado automáticamente a partir de la auditoría de código ObSave v1.0.35.